



dcc CIENCIAS DE LA COMPUTACIÓN
UNIVERSIDAD DE CHILE

MINERÍA DE DATOS

Clase: Clasificación III
(SVM, bagging, boosting)

Profesores: Cinthia Sánchez y Cristian Llull

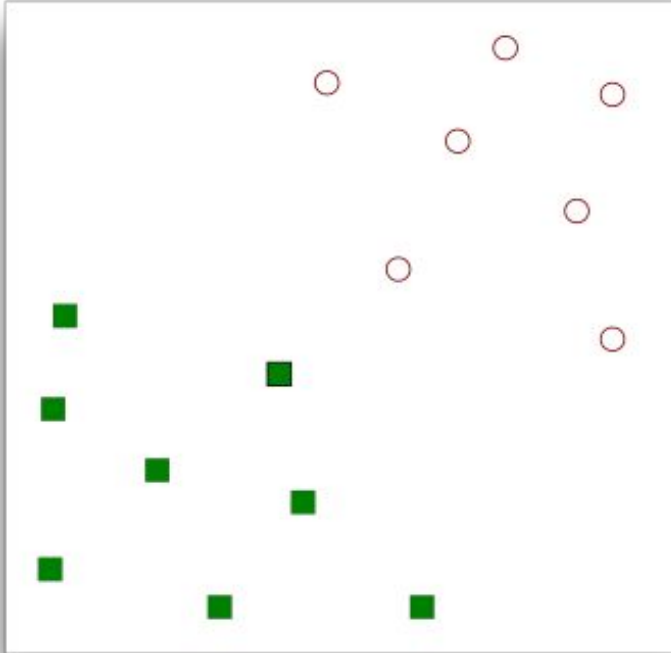
Support Vector Machines

Se formula el problema de clasificación como un problema de optimización.

Encontrar un hiperplano lineal (decision boundary).

- Representa una frontera de decisión usando un subconjunto de ejemplos de entrenamiento, conocidos como support vectors.
- Trabaja bien con datos de alta dimensionalidad y evita la "*curse of dimensionality*"

Support Vector Machines

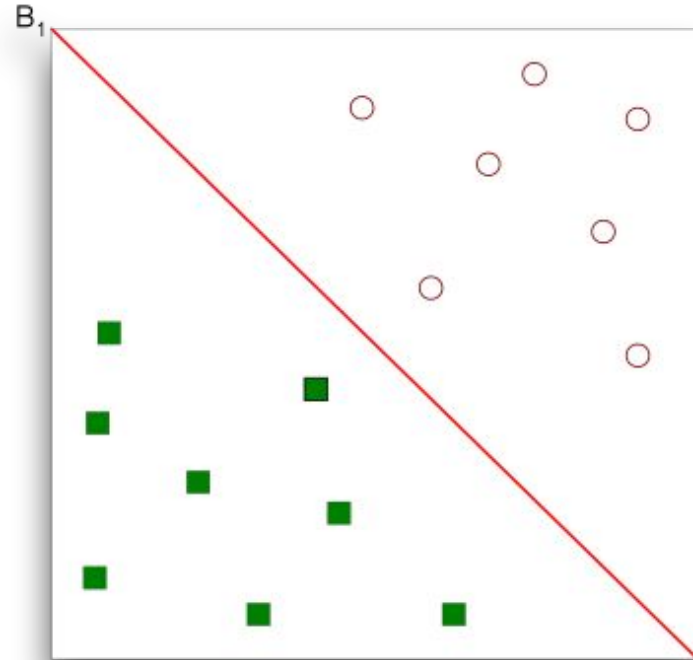
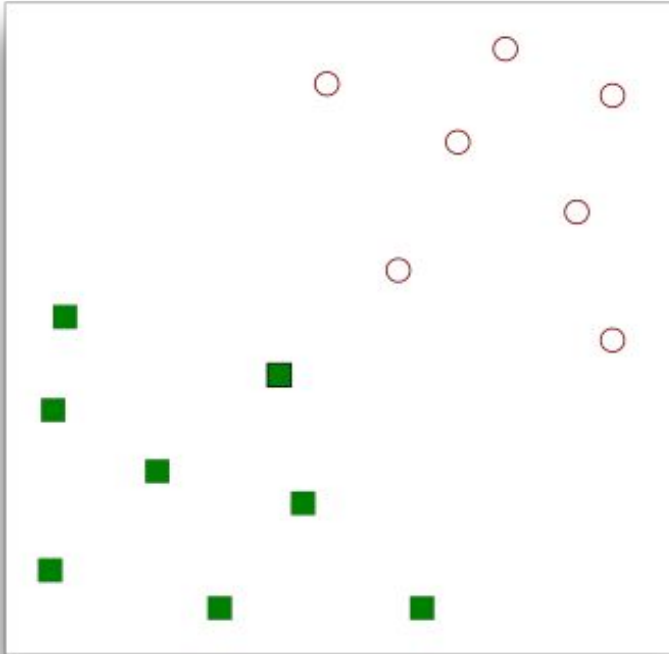


Ejemplo: Tenemos un dataset de n -dimensiones (proyectado a 2).

Podemos decir que estos datos son linealmente separables (separables por un hiperplano)

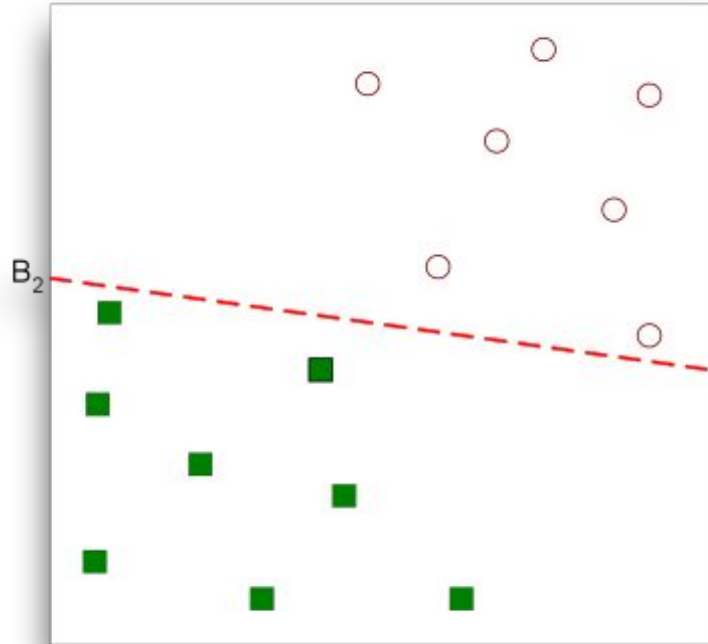
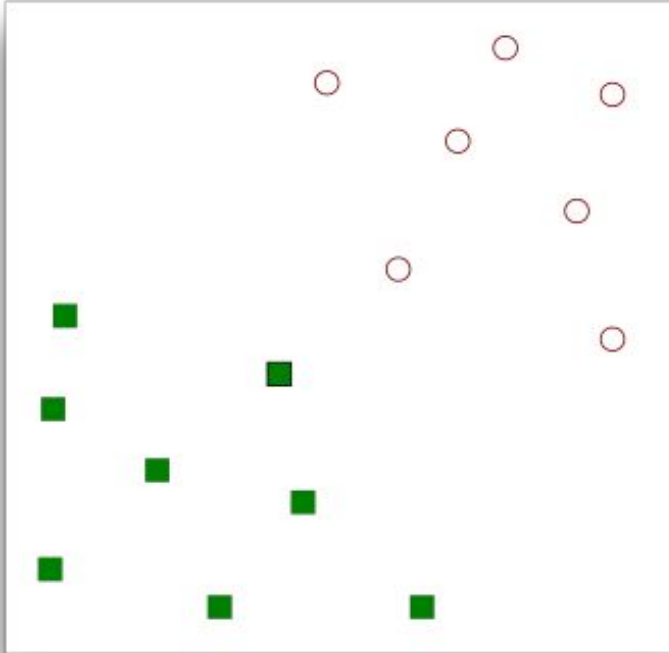
- Encontrar un hiperplano lineal (*decision boundary*) que separe los ejemplos positivos de los negativos.

Support Vector Machines



Una posible solución

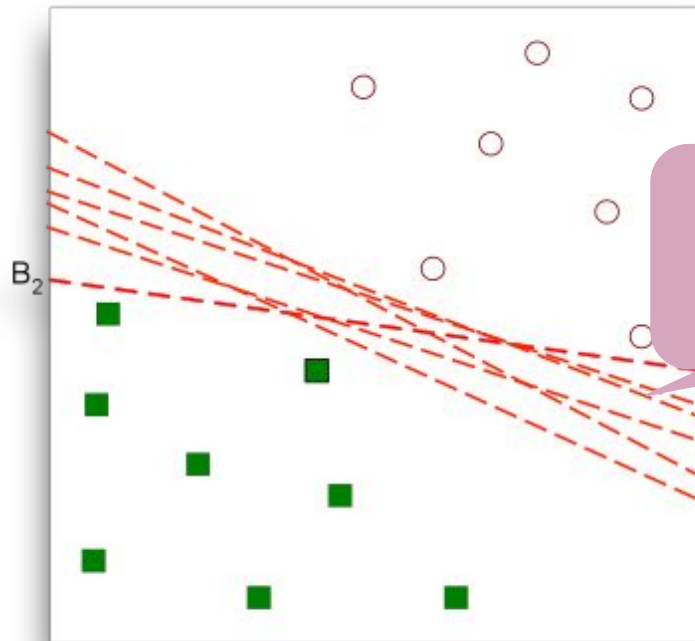
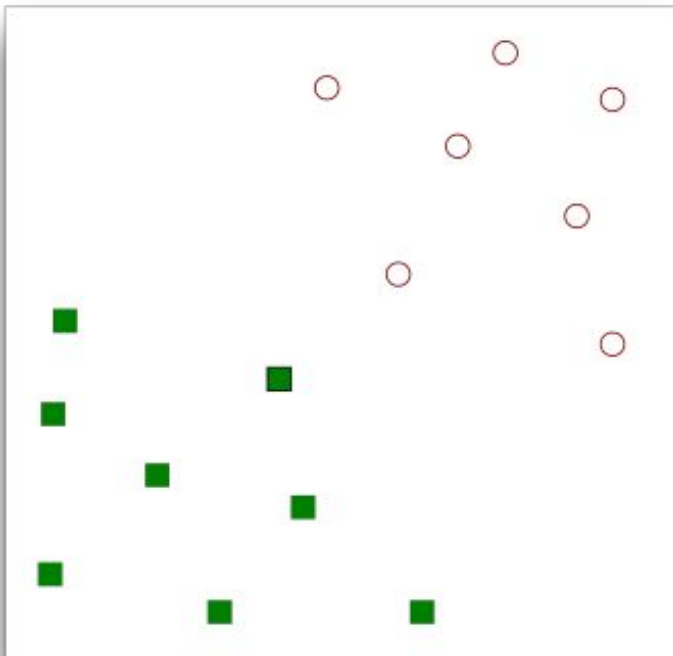
Support Vector Machines



Otra posible solución

Support Vector Machines

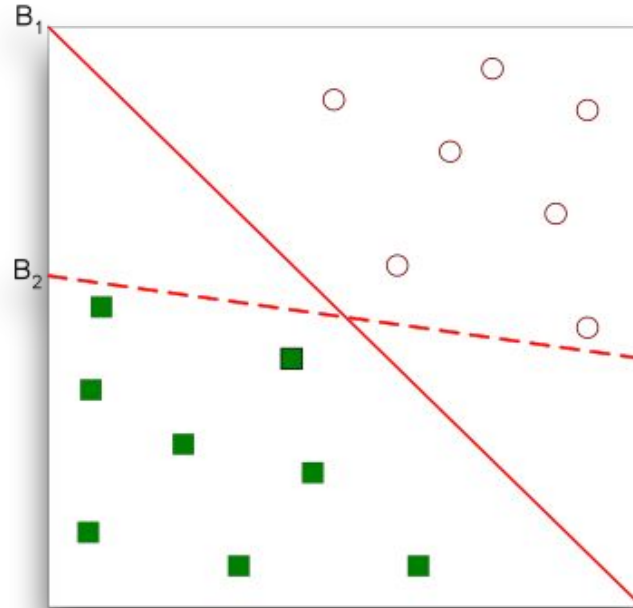
Este modelo puede extenderse a datos que no son linealmente separables.



SVM se basa en aprender a encontrar el plano marginal maximal

¡Infinitas soluciones!

Support Vector Machines



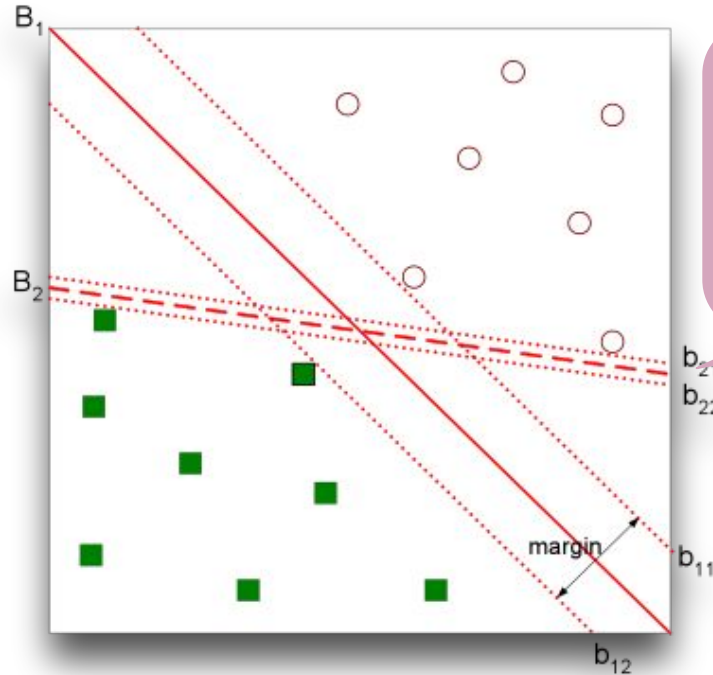
¿Cuál es mejor? ¿ B_1 o B_2 ?
¿Cómo definimos qué es mejor?

Support Vector Machines

Encontrar un hiperplano que **maximice** el margen de entrenamiento (menores errores de generalización)

- El margen del hiperplano es la distancia que hay de los puntos positivos y negativos más cercanos al hiperplano.

Entre más ancho el margen, mayor el poder de generalización.
=> B1 es mejor que B2



Menor margen hace que sea más susceptible a ruido y perturbaciones, lo que puede llevar a overfitting y mala generalización.

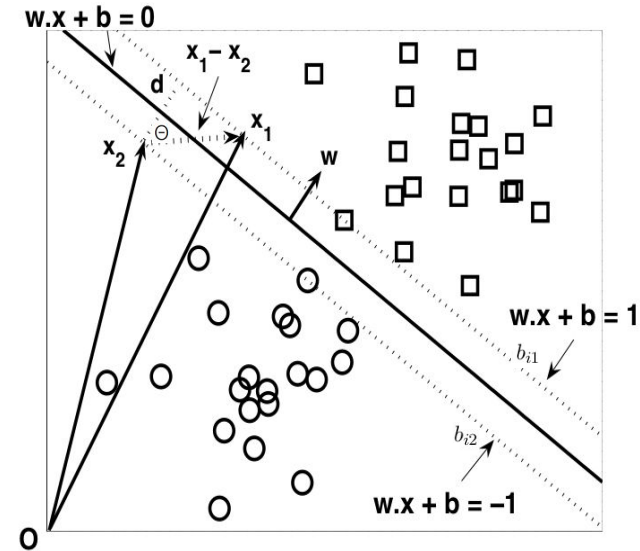
Support Vector Machines

Un SVM lineal es un clasificador que busca un hiperplano con el máximo margen.

- Sea un problema de clasificación binaria con N ejemplos, cada ejemplo se representa como la tupa x_i, y_i ($i=1,2,\dots,N$), donde x_i es un vector de d dimensiones ($x_{i1}, x_{i2}, \dots, x_{id}$)^T e $y_i \in \{-1, 1\}$ (ejemplos negativos y positivos).
- El límite de decisión de un clasificador lineal se escribe de la siguiente manera:

$$\mathbf{w} \cdot \mathbf{x} + b = 0,$$

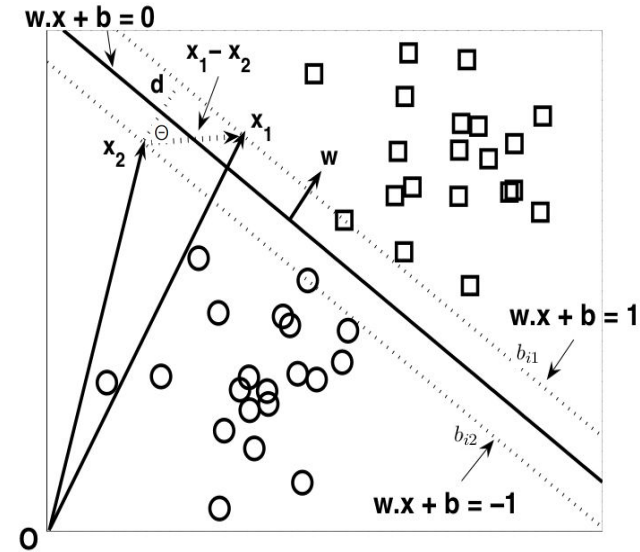
- Donde $\mathbf{w}$ y b son parámetros del modelo.



Support Vector Machines

Consiste en:

1. Formular el margen en función de los parámetros.
2. Formular un problema de optimización que permita encontrar el hiperplano de máximo margen.



Support Vector Machines

- El **hiperplano** $\mathbf{w} \cdot \mathbf{x} + b = 0$ separa los ejemplos positivos (cuadrados) de los negativos (círculos)
- Cualquier ejemplo que se encuentre en el límite de decisión debe satisfacer la ecuación del hiperplano.
- Por ejemplo, sean $\mathbf{x}_a$ y $\mathbf{x}_b$ dos puntos localizados en el límite de decisión:

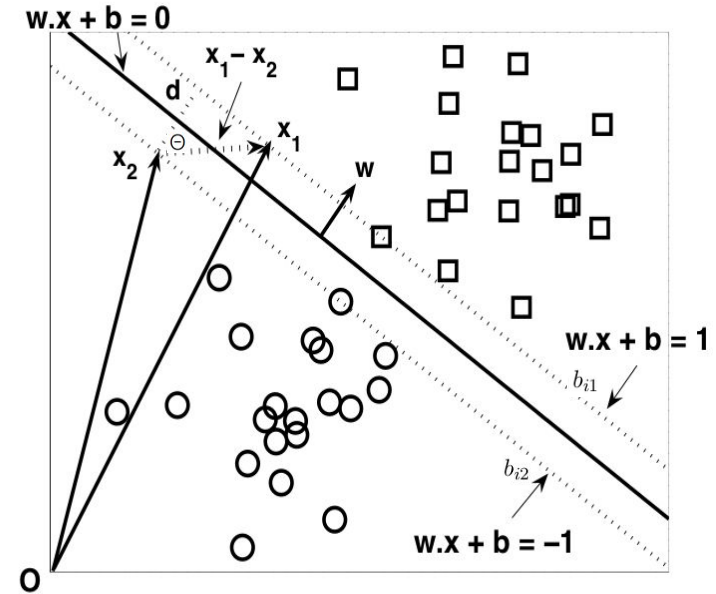
$$\mathbf{w} \cdot \mathbf{x}_a + b = 0,$$

$$\mathbf{w} \cdot \mathbf{x}_b + b = 0.$$

- Si restamos las dos ecuaciones obtenemos:

$$\mathbf{w} \cdot (\mathbf{x}_b - \mathbf{x}_a) = 0,$$

- Cómo $\mathbf{x}_b - \mathbf{x}_a$ es paralelo al límite de decisión, entonces $\mathbf{w}$ es perpendicular al hiperplano (el producto punto de dos vectores ortogonales es cero).



Support Vector Machines

- Para cualquier cuadrado x_s localizado **sobre el límite de decisión**, se cumple que:

$$\mathbf{w} \cdot \mathbf{x}_s + b = k,$$

donde $k > 0$.

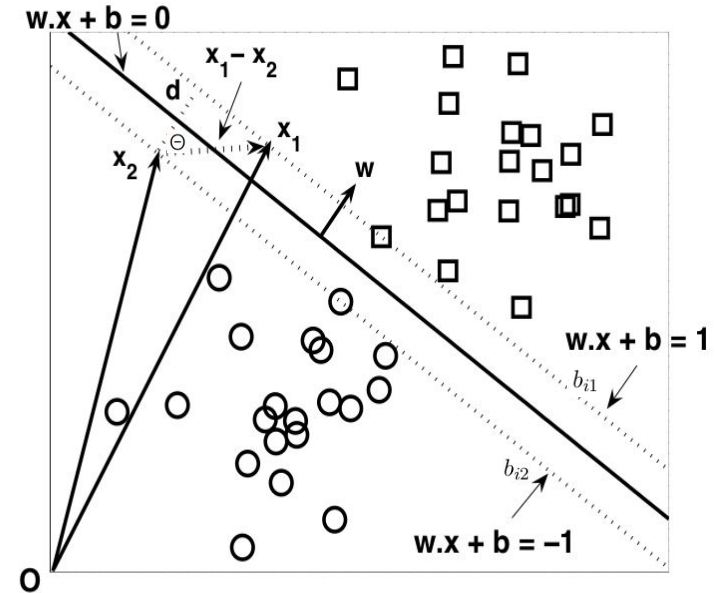
- De manera análoga, para cualquier círculo x_c localizado **bajo el límite de decisión**, se cumple que:

$$\mathbf{w} \cdot \mathbf{x}_c + b = k',$$

done $k' < 0$.

- Si etiquetamos todos los cuadrados como la clase positiva +1 y todos los círculos como la clase negativa -1, podemos predecir la etiqueta $\mathbf{y}$ para cualquier ejemplo de test $\mathbf{z}$ de la siguiente manera:

$$y = \begin{cases} 1, & \text{if } \mathbf{w} \cdot \mathbf{z} + b > 0; \\ -1, & \text{if } \mathbf{w} \cdot \mathbf{z} + b < 0. \end{cases}$$



El Margen de un Clasificador Lineal

- Podemos re-escalar los parámetros w y b del límite de decisión de tal manera que los hiperplanos paralelos b_{i1} y b_{i2} puedan expresarse de la siguiente manera:

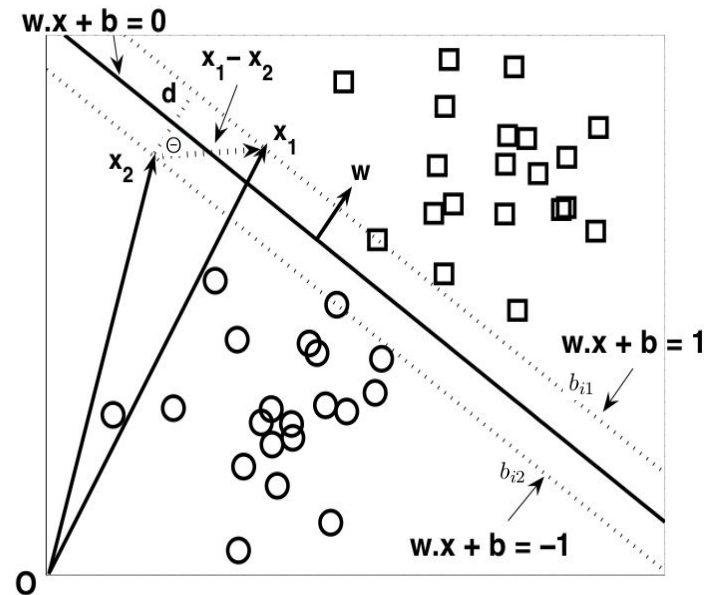
$$b_{i1} : \mathbf{w} \cdot \mathbf{x} + b = 1,$$

$$b_{i2} : \mathbf{w} \cdot \mathbf{x} + b = -1.$$

- El margen del límite de decisión se calcula como la distancia entre estos dos hiperplanos.
- Calculamos la distancia del cuadrado x_1 y el círculo x_2 . Esto se hace sustituyendo x_1 y x_2 en las ecuaciones de b_{i1} y b_{i2} respectivamente. Lo que nos da:

$$1) \mathbf{w} \cdot \mathbf{x}_1 + b = 1 \text{ y } 2) \mathbf{w} \cdot \mathbf{x}_2 + b = -1.$$

- Si sustraemos la segunda ecuación de la primera obtenemos $\mathbf{w} \cdot (\mathbf{x}_1 - \mathbf{x}_2) = 2$



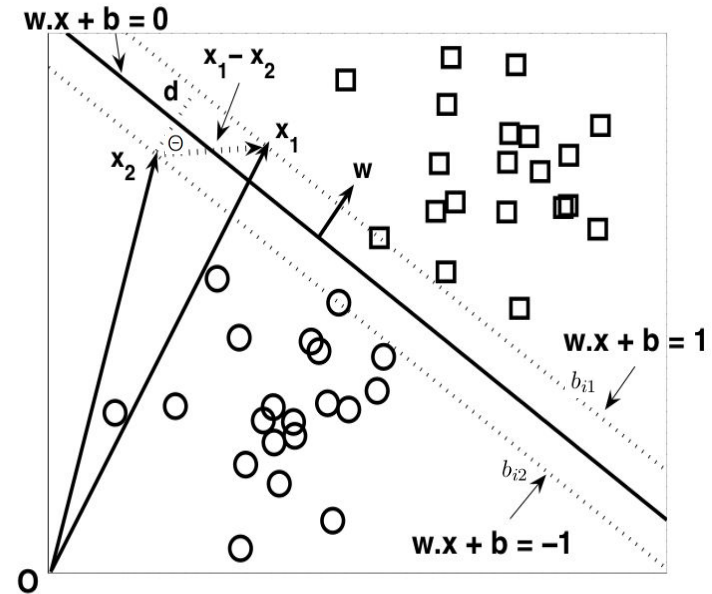
El Margen de un Clasificador Lineal

- El producto punto de dos vectores ($a \cdot b$) se puede representar como la norma $\|a\| \cdot \|b\| \cdot \cos(\Theta)$. Es decir, multiplicar sus magnitudes (normas) por el coseno del ángulo que forman entre ellos.
- Tenemos entonces que $\|w\| \cdot \|x_1 - x_2\| \cdot \cos(\Theta) = 2$
- Si miramos la figura podemos ver que $\|x_1 - x_2\| \cdot \cos(\Theta) = d$.
- Entonces

$$\|w\| \times d = 2$$

$$\therefore d = \frac{2}{\|w\|}.$$

Queremos maximizar este margen



¡ Encontramos una expresión del margen que depende de w !

Aprendiendo una SVM Lineal

La fase de **entrenamiento de una SVM lineal** implica la estimación de los parámetros w y b del límite de decisión a partir de los datos de entrenamiento.

Los parámetros deben elegirse de manera que se cumplan las dos siguientes condiciones:

$$\begin{aligned}w \cdot x_i + b &\geq 1 \text{ if } y_i = 1, \\w \cdot x_i + b &\leq -1 \text{ if } y_i = -1.\end{aligned}$$

- Estas condiciones imponen el requisito de que todas las instancias de entrenamiento de la clase $y = 1$ (los cuadrados) deben estar situadas en o **sobre el hiperplano $w \cdot x + b = 1$** .
- Mientras que las instancias de la clase $y = -1$ (los círculos) deben estar situadas **en o debajo del hiperplano $w \cdot x + b = -1$** .

Aprendiendo una SVM Lineal

- Maximizar el margen equivale a minimizar la siguiente función objetivo:

$$f(\mathbf{w}) = \frac{\|\mathbf{w}\|^2}{2}.$$

$$\max_w \frac{2}{\|w\|} \Leftrightarrow \min_w \frac{\|w\|^2}{2}$$

- La tarea de aprendizaje en SVM puede formalizarse como el siguiente problema de optimización

$$\begin{aligned} & \min_{\mathbf{w}} \frac{\|\mathbf{w}\|^2}{2} \\ & \text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, N. \end{aligned}$$

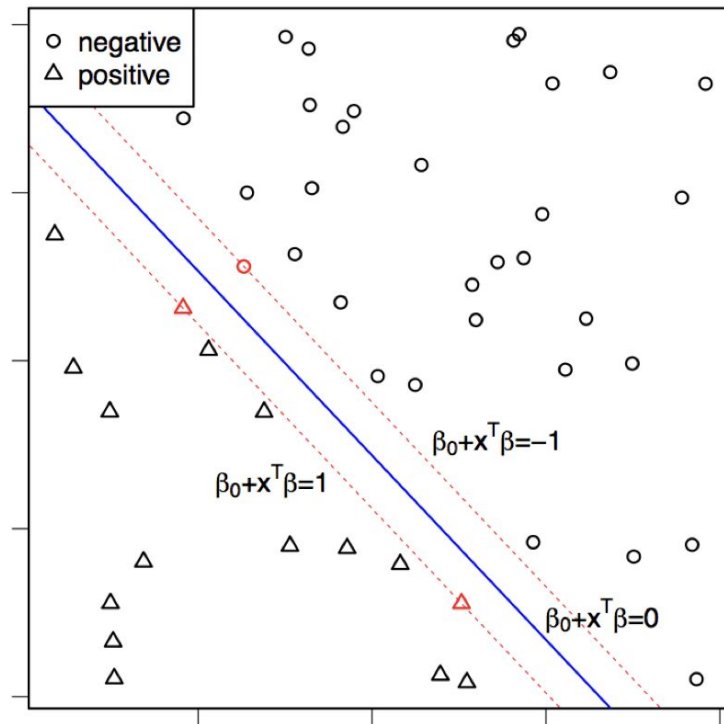
- Dado que la función objetivo es cuadrática y las restricciones son lineales para los parámetros w y b , esto se conoce como un **problema de optimización convexa**.

Hiperplano óptimo: puntos de soporte

- El hiperplano está definido por una combinación lineal de puntos en el borde del margen (puntos de soporte).
- ¡La solución sólo depende de los puntos de soporte y no de todos los datos!
- Por lo que podemos escribir

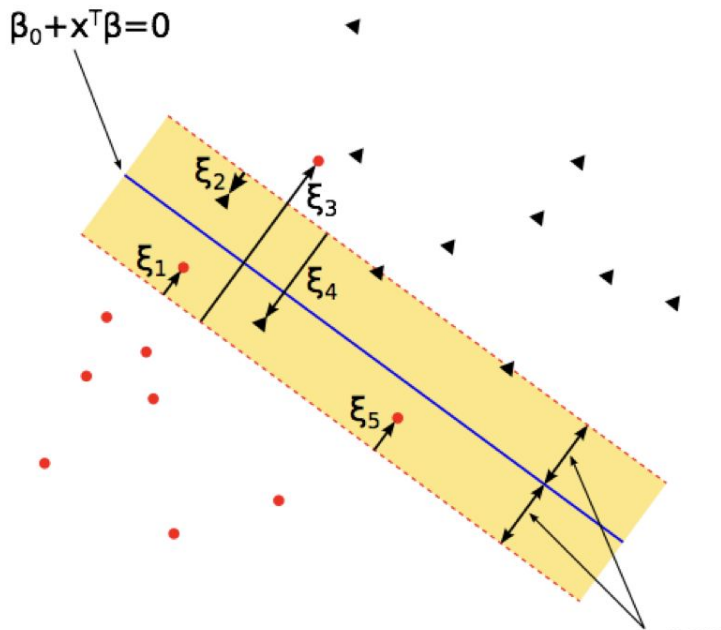
$$\beta = x^T \alpha$$

- donde $\alpha_i = 0$ cuando no es un punto de soporte.



Aprendiendo una SVM Lineal (más realista)

- Para hacerlo "más realista" (Soft Margin) porque los datos casi nunca son perfectamente separables, introducimos lo siguiente.
- Introducir para cada ejemplo una variable auxiliar (de Holgura) $\xi_i \geq 0$
- ξ_i cantidad relativa, respecto al margen, por la cual la predicción está en el lado incorrecto
- Si la clasificación es correcta $\xi_i = 0$
- Si $0 < \xi_i \leq 1$ el punto está entre el margen y el lado correcto
- Clasificaciones incorrectas ocurren si $\xi_i > 1$



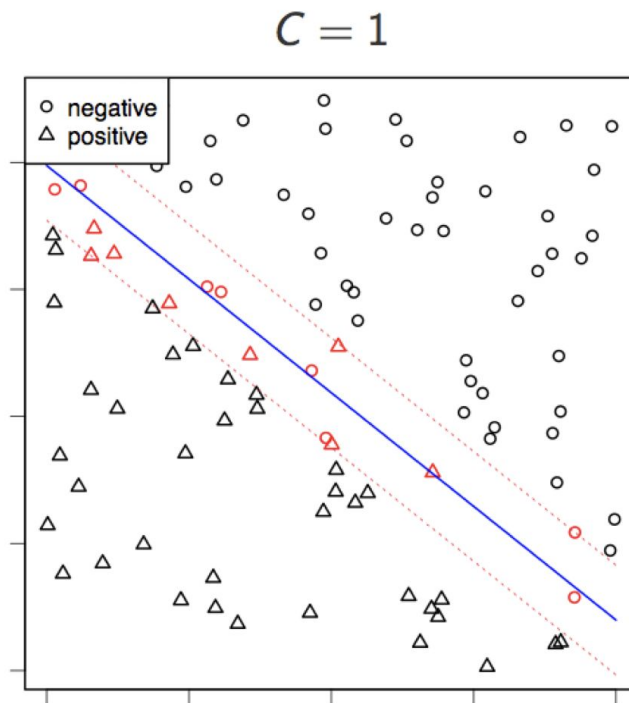
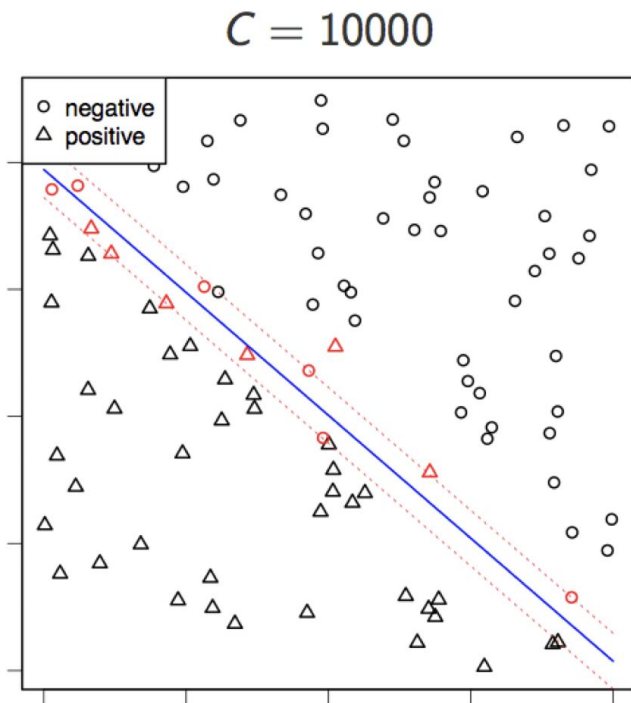
Aprendiendo una SVM Lineal (más realista)

$$\min_{\beta_0, \beta} \frac{1}{2} |\beta|^2 + C \sum_{i=1}^n \xi_i$$

sujeto a $\xi_i \geq 0, y_i(\beta_0 + x^T \beta) \geq 1 - \xi_i$

- El parámetro $C > 0$ representa el trade-off entre la penalización de la variable auxiliar y el margen.
- Si $C = \infty$ el resultado es equivalente al hiperplano óptimo
 - C muy alto: el modelo hará el margen muy estrecho con tal de minimizar errores.
 - C bajo: prefiere una frontera más ancha y suave que generalice mejor.

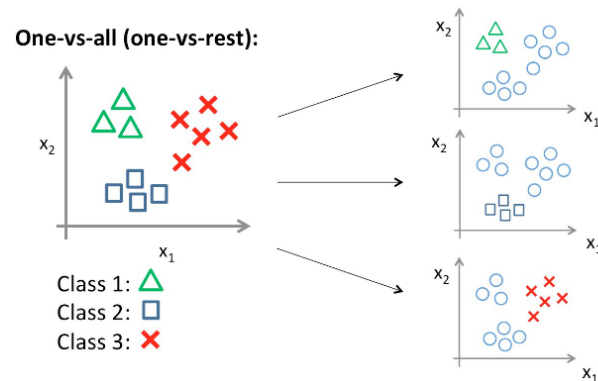
Aprendiendo una SVM Lineal (más realista)



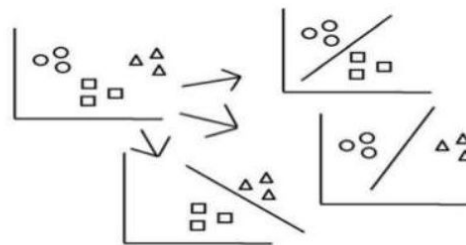
Extensión de SVM a Multiclase

SVM es estrictamente binario, se extiende dividiendo el problema mediante dos estrategias principales:

- **Uno contra el Resto (One-vs-Rest):** Si hay N clases, entrena N modelos. Cada SVM aprende a separar una clase específica de todas las demás juntas. Para predecir, elige la clase del modelo que dé la puntuación más alta (mayor confianza).
- **Uno contra Uno (One-vs-One):** Entrena un SVM para cada par posible de clases. Si hay 3 clases (A, B, C), entrena A vs B, A vs C, y B vs C. Para predecir, gana la clase con más votos en total.

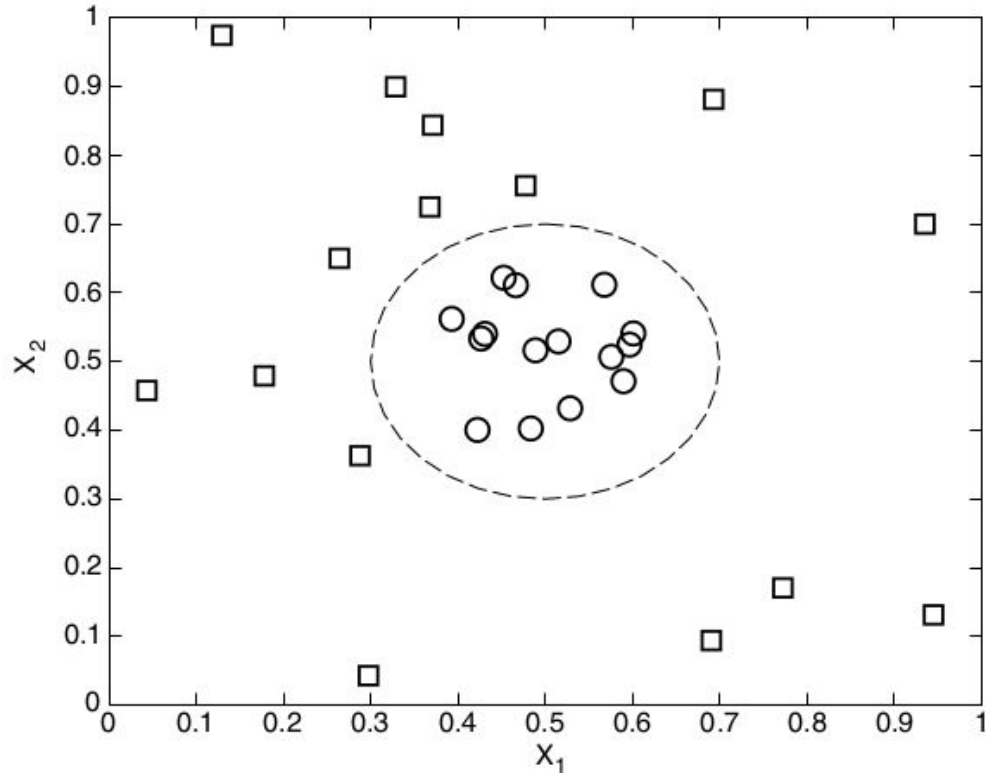


One-vs-One (OVO)

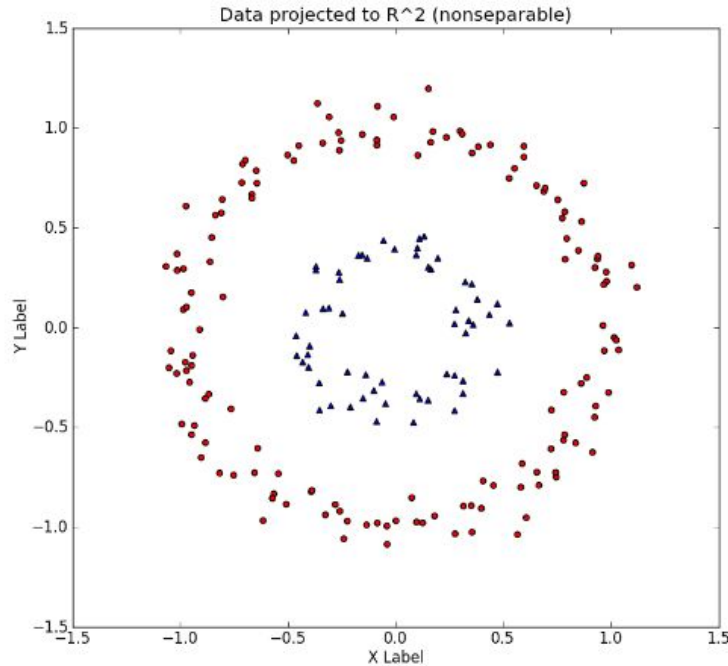


Support Vector Machines No-Lineales

- El diagrama muestra un ejemplo de un dataset bidimensional compuesto por cuadrados ($y = 1$) y círculos ($y = -1$).
- Todos los círculos están agrupados cerca del centro del diagrama y todos los cuadrados se distribuyen más lejos del centro.
- Este problema no se puede resolver con una SVM lineal.



Support Vector Machines No-Lineales



¿qué transformación podríamos hacer para que este problema sea linealmente separable?

source: http://www.eric-kim.net/eric-kim-net/posts/1/kernel_trick.html

Support Vector Machines No-Lineales

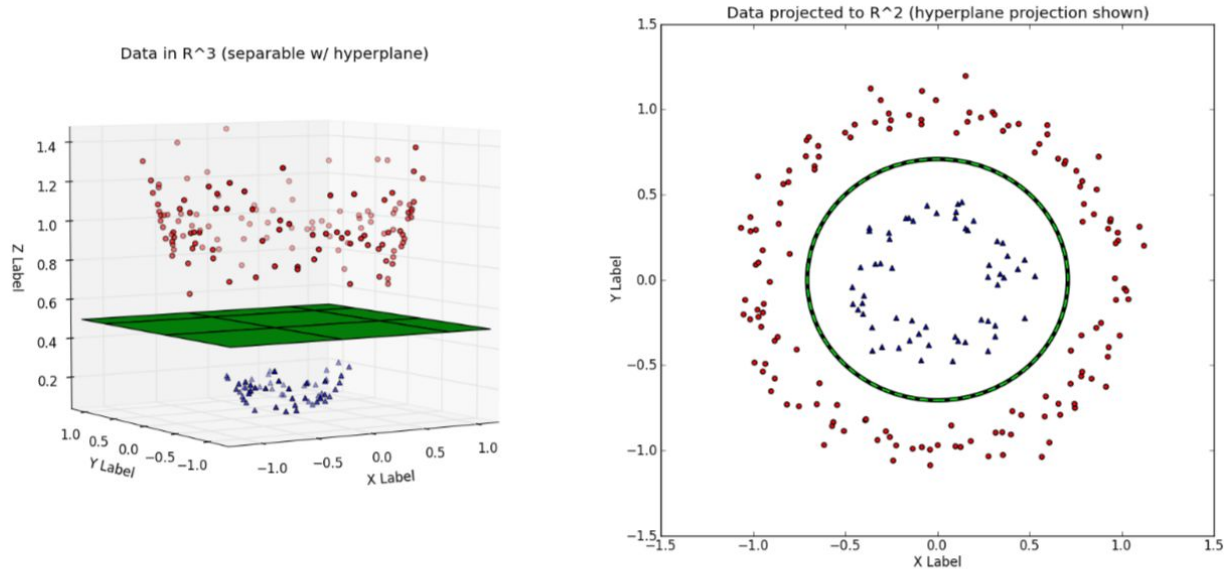


Figure 6: (Left) The decision boundary $\vec{w}$ shown to be linear in $\mathbb{R}^3$. (Right) The decision boundary $\vec{w}$, when transformed back to $\mathbb{R}^2$, is nonlinear.

source: http://www.eric-kim.net/eric-kim-net/posts/1/kernel_trick.html

Support Vector Machines No-Lineales

Podríamos aplicar una **transformación no lineal Φ** para mapear los datos de su espacio de original a un nuevo espacio (de más dimensiones) donde el límite de decisión se vuelva lineal.

Supongamos que elegimos la siguiente transformación que transforma de 2 dimensiones a 5 dimensiones:

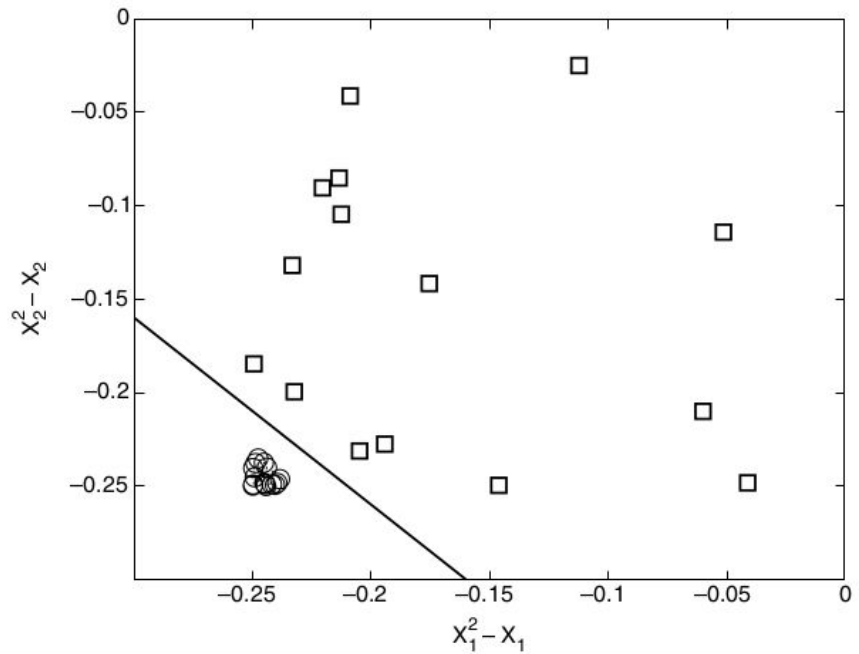
$$\Phi : (x_1, x_2) \longrightarrow (x_1^2, x_2^2, \sqrt{2}x_1, \sqrt{2}x_2, 1).$$

En este espacio transformado si es posible encontrar parámetros $\mathbf{w} = (w_0, w_1, \dots, w_4)$ que separen linealmente los datos:

$$w_4x_1^2 + w_3x_2^2 + w_2\sqrt{2}x_1 + w_1\sqrt{2}x_2 + w_0 = 0.$$

Support Vector Machines No-Lineales

- La figura muestra que en el espacio transformado se puede construir un límite de decisión lineal para separar las clases.
- Un problema potencial de este enfoque es que puede sufrir de la **maldición la dimensionalidad**: para datos de alta dimensión muchas técnicas de data mining no escalan o no funcionan bien.
- Mostraremos cómo una SVM no lineal evita este problema usando un truco llamado **Kernel Trick**.



SVM kernel trick

- Calcular explícitamente la transformación no-lineal puede ser costoso.
- Sin embargo recordar que, β es una combinación lineal de los puntos de soporte, ie $\beta = x^T \alpha$ y $\beta_j = \sum_{i=1}^n \alpha_i x_{ij}$ donde $\alpha = 0$ si x_i no es de soporte.
- El hiperplano separatriz se puede formular como:

$$f(\mathbf{x}_0) = \beta_0 + \sum_{j=1}^m x_{0j} \beta_j = \beta_0 + \sum_{i=1}^n \alpha_i \mathbf{x}_i^T \mathbf{x}_0$$

- Aplicando la transformación

$$f(\mathbf{x}_0) = \beta_0 + \sum_{i=1}^n \alpha_i \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_0)$$

SVM kernel trick

- Definición (función kernel)

$$K(x, x') = \phi(x)^T \phi(x')$$

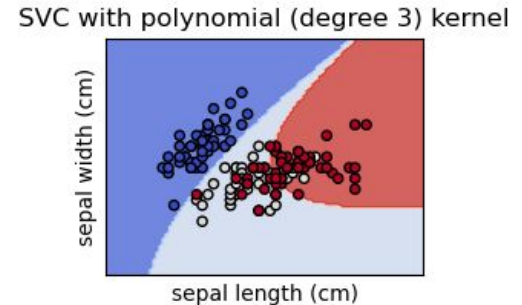
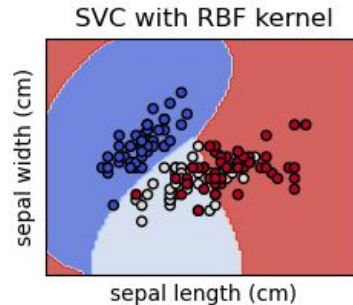
- Definición (kernel SVM)

$$f(x_0) = \beta_0 + \sum_{i=1}^n \alpha_i K(x_i, x_0)$$

- El “truco del kernel”: no se requiere transformar los datos.
- ¡No se requiere una forma explícita para ϕ !

Support Vector Machines No-Lineales

- Se pueden especificar diferentes funciones de kernel para la función de decisión. Se proporcionan kernels comunes, pero también es posible especificar kernels personalizados.
 - Polinomial
 - Exponencial
 - Radial Basis Function (RBF)
 - etc.

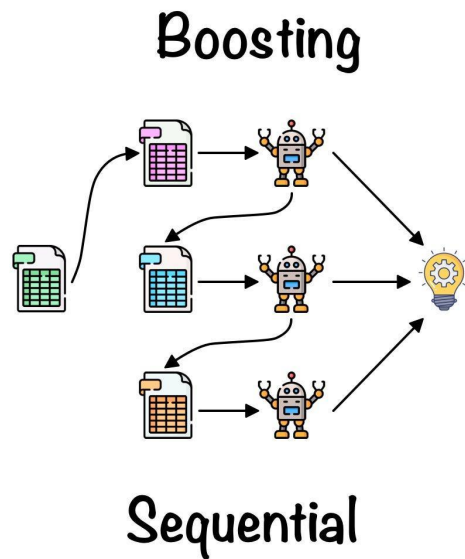
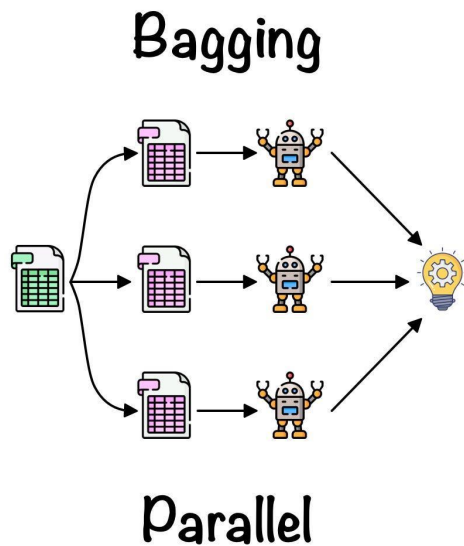


Resumen SVM

- La formulación de SVM presentada en esta clase se limita a problemas de clasificación binaria. Existen adaptaciones para trabajar con múltiples clases.
- El problema de aprendizaje de una SVM se formula como un problema de optimización convexa en donde hay algoritmos eficientes para encontrar el óptimo global. Otros métodos de clasificación como los árboles de decisión y las redes neuronales tienden a encontrar óptimos locales.
- La SVM optimiza explícitamente la capacidad de generalización al maximizar el margen del límite de decisión.
- En una SVM el usuario debe ajustar hiper-parámetros, como el tipo de función de Kernel y el costo C para las variables de holgura (esto puede ser caro).
- La gran limitación de las SVMs es que no escalan bien para datasets masivos.

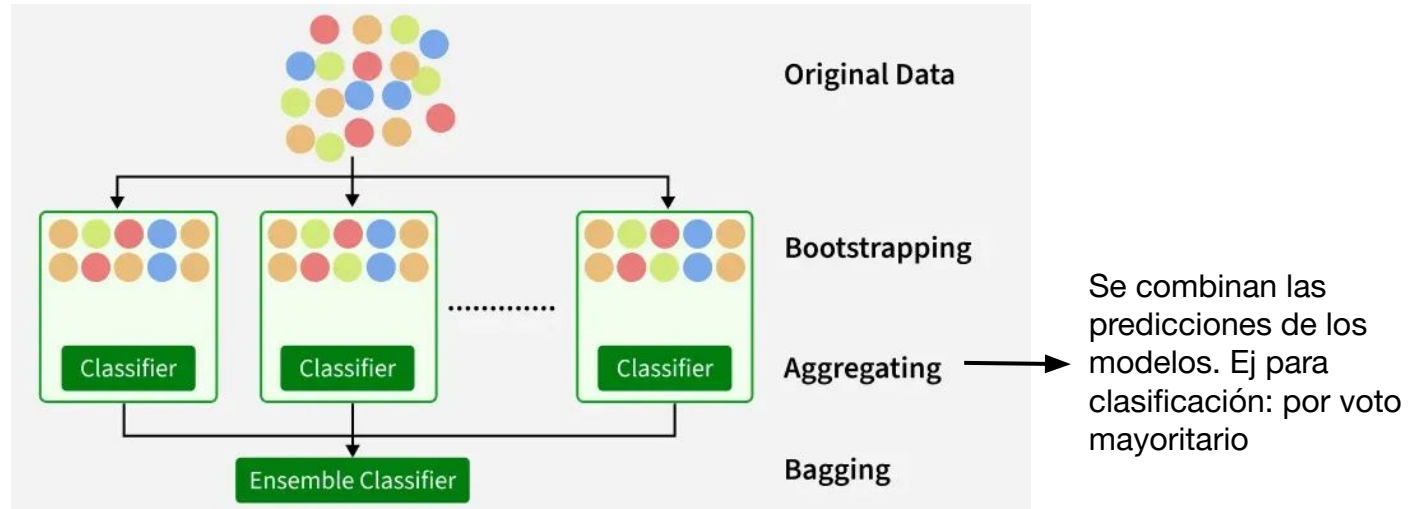
¿Cómo combinar clasificadores?

- Supongamos que tenemos más de un clasificador ¿cómo combinarlos?



Bagging (Bootstrap Aggregating)

- “La sabiduría de la multitud”
- Si tuviéramos N clasificadores ligeramente diferentes (no-correlacionados) la clasificación tendría mejor generalización.
- Los árboles son el ejemplo más frecuente, donde se entrenan N árboles con un subconjunto ligeramente diferente de datos. Ej: Random Forest



Ejemplo: Riesgo de crédito bancario



1. Muestreo de Clientes

A partir de la base de datos de 10,000 clientes, se generan **10** grupos aleatorios. Un mismo cliente puede aparecer en varios grupos.



2. Modelos Paralelos

Se entrena un Árbol de Decisión separado para cada grupo de clientes. Tenemos **10** 'expertos' observando distintos fragmentos de información.



3. Votación Final

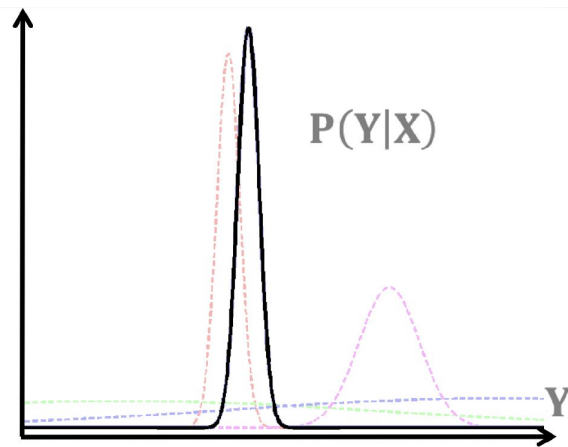
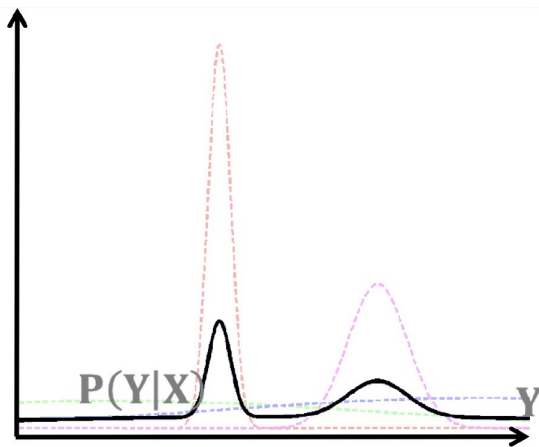
Llega un nuevo solicitante. **8** árboles predicen 'Buen pagador' y **2** predicen 'Riesgo'. Por mayoría, se aprueba el crédito.

Bagging: votación

- Al terminar el bagging (o bootstrap) tendremos múltiples respuestas.

$$P(Y|X) = \frac{1}{T} \sum_{t=1}^T P_t(Y|X)$$

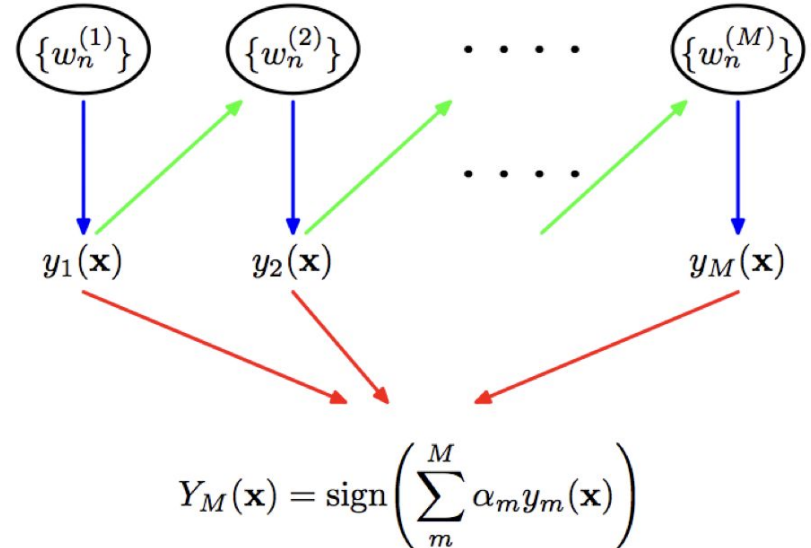
$$P(Y|X) = \frac{1}{Z} \prod_{t=1}^T P_t(Y|X)$$



Boosting

- Aprende de los errores del pasado de manera secuencial.
- Supongamos que tenemos clasificadores que son sólo ligeramente mejores que una respuesta aleatoria (clasificadores débiles), ¿cómo construir un buen clasificador? (fuerte)
- A diferencia de bagging, los modelos se entrenan uno tras otro, no en paralelo.
- Cada modelo se enfoca en corregir los errores del modelo anterior, dándole mayor peso a estos casos.
- Ejemplo: **AdaBoost**, XGBoost, GradientBoost

Schematic illustration of the boosting framework. Each base classifier $y_m(\mathbf{x})$ is trained on a weighted form of the training set (blue arrows) in which the weights $w_n^{(m)}$ depend on the performance of the previous base classifier $y_{m-1}(\mathbf{x})$ (green arrows). Once all base classifiers have been trained, they are combined to give the final classifier $Y_M(\mathbf{x})$ (red arrows).



Ejemplo: Diagnóstico médico



1. Modelo Débil e Identificación de Fallos

Se entrena un modelo inicial simple (ej. árbol). Identifica la mayoría, pero no las anomalías más complejas.



2. Ponderación Secuencial de Errores

A los pacientes mal diagnosticados se les asigna un 'peso' mayor. El Modelo 2 se entrena secuencialmente para corregirlos.



3. Combinación Ponderada para Predicción Final

Tras iterar, se combinan todos los modelos vía votación ponderada. El sistema 'fuerte' resultante es altamente robusto y preciso.